Reinforcement Learning Assignment 3: Advanced Actor Critic - PPO and/or SAC Report

Yutao Liu(s4213211) [1] **Nael Belhaj Haddou (s4431278)** [1]

Abstract

1. Introduction

In the first assignment, we explored Q-Learning and its extension to Deep Q-Networks (DQN), where the focus was on approximating value functions through neural networks to overcome scalability limitations of tabular methods. However, these approaches primarily relied on value-based methods and hard-coded policies, limiting their flexibility.

In the second assignment, we transitioned to policy-based methods by implementing REINFORCE and basic Actor-Critic (AC) algorithms. These approaches introduced parametrized policies and demonstrated how combining value estimation with policy optimization could improve learning efficiency. Nevertheless, instability during policy updates remained a significant challenge, particularly for AC and A2C algorithms.

Building upon these foundations, this third assignment investigates Proximal Policy Optimization (PPO), a modern actor-critic algorithm designed to address the stability issues of previous methods. PPO employs a clipped surrogate objective to constrain policy updates, leading to more reliable and efficient training. In this report, we present a theoretical overview of PPO, detail our implementation on the Cart-Pole environment, and compare its performance with earlier methods.

2. Theory

2.1. Proximal Policy Optimisation(PPO)

In previous repeated experiments, we observed that the standard Actor-Critic algorithm (such as A2C) has a problem of too large policy changes when updating the policy, which directly leads to instability in training. (Schulman et al., 2017) proposed a Proximal Policy Optimization (PPO) method, which introduced a clipped surrogate objective... which introduced clipped surrogate objective that penalizes policy changes exceeding a certain threshold, to ensure that each policy iteration does not deviate too far from the original policy and maintain learning stability.

Below is pseudo-code describing the algorithm as it's been implemented here.

Algorithm 1 Proximal Policy Optimization Algorithm

Input: actor $\pi_\theta(a|s)$, critic $V_\phi(s)$
Parameters: discount factor γ, clipping parameter ϵ, number of epochs K, number of mini-batches N, entropy coefficient ϵ, decay rateλ
for each iteration **do**
  Collect trajectories by running policy π_θ in environment
  Estimate advantage $\hat{A}(s,a)$ using GAE(λ) and compute returns R
  for K epochs **do**
    for N mini-batches **do**
      Sample mini-batch of transitions (s,a,r,s')
      Compute probability ratio: $r(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$
      Compute clipped objective: $\mathcal{L}_a = \min\left(r(\theta)\hat{A}, \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)\hat{A}\right)$
      compute policy entropy S
      Update actor by maximizing $\mathcal{L}_a + \epsilon \times S$
      Compute value loss: $\mathcal{L}_c = (V_\phi(s) - R)^2$
      Update critic by minimizing $\mathcal{L}_c$
    end for
  end for
end for

2.2. Loss Function

PPO employs two distinct loss functions: one for optimizing the policy (actor) and another for training the value function (critic). The clipped surrogate objective builds on the standard policy gradient formulation described in (Sutton & Barto, 2018), modifying it to improve update stability.

The policy loss uses a clipped surrogate objective, defined as:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right], \tag{1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ represents the probability ratio between the new and old policies, $\hat{A}_t$ is the estimated advantage at time step t, and ϵ is the clipping parameter (typically set between 0.1 and 0.2).

This objective encourages policy updates only when they do not cause drastic deviations from the current policy, thereby maintaining training stability.

In addition to policy optimization, PPO also trains a value function to approximate expected returns. The critic loss minimizes the mean squared error (MSE) between the predicted state value and the empirical return:

$$L^{\text{VF}}(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - R_t \right)^2 \right], \tag{2}$$

where $V_\phi(s_t)$ denotes the estimated value from the critic network with parameters ϕ, and R_t represents the empirical return computed based on collected rewards.

Optionally, PPO may combine the policy loss and the value loss (and possibly an entropy bonus) into a single composite objective for joint optimization.

2.3. Comparison to A2C

- **Policy Objective:**
  - A2C uses the objective $\log \pi(a|s) \times$ Advantage.
  - PPO uses a clipped surrogate objective to constrain policy updates.
- **Update Stability:**
  - A2C can suffer from unstable updates when the policy changes significantly.
  - PPO restricts updates to maintain stability, generally leading to more reliable training.
- **Training Mode:**
  - A2C typically updates after each batch of transitions.
  - PPO performs multiple epochs of updates over the same batch of data.
- **Use of GAE:**
  - Our A2C implementation does not use Generalized Advantage Estimation (GAE).
  - PPO typically benefits from using GAE to reduce variance in advantage estimation.
- **Engineering Tricks:**
  - A2C uses minimal engineering tricks.
  - PPO incorporates extensive tricks such as mini-batching, advantage normalization, and entropy bonus.

In conclusion, PPO improves training stability and sample efficiency compared to A2C by limiting the extent of policy updates in each iteration.

2.4. Engineering Tricks

This implementation uses generalized advantage estimation (GAE) to compute advantages, this helps reduce variance in advantage estimation. The formula used in GAE is shown in equation 3 where t is a time step, s_t and r_t are respectively the state and reward at time step t, γ is the discount factor, λ is the decay rate and V is the value function. The advantage obtained from this is then normalized to help with variance.

$$GAE(t, \gamma, \lambda) = r_t + \gamma V(s_{t+1}) - V(s_t) + \lambda\gamma GAE(t+1, \gamma, \lambda) \tag{3}$$

The second engineering trick used here is mini-batching. Each epoch, the data is broken into smaller mini-batches, this breaks the connection between samples by shuffling them and helps with performances by lightening the load on memory.

The third engineering trick is an entropy bonus added to policy loss. The policy used here is stochastic, thus to encourage exploration the policy is encouraged to stay somewhat random by adding entropy to the loss. This prevents the agent from converging prematurely.

3. Experiment

All the results showcased here are the mean and standard deviation of performance over 5 runs of 1 million timesteps each.

Furthermore, in all subsequent experiments the same hyperparameter configuration is used for PPO. The networks used for the policy and value function both have 2 hidden layers and 32 neurons per layer, the optimization is done using ADAM with a learning rate of 0.0003. Gradient descent is performed every 512 timesteps, the data is passed over 4 times and broken into 8 mini-batches and the PPO clip coefficient is set to 0.2. During training the entropy coefficient is set to 0.01, a gamma of 0.99 is used and the lambda used in GAE is 0.95.

3.1. Environments

Three environments are studied here. The first is cartpole it tasks an agent with balancing a pole on top of a cart by moving it left and right. Episodes have the agent start with the pole at an angle of 0 and the cart at a position in the range ± 0.05 and with both not moving. Episodes terminate when the cart gets more than 2.4 units away from the center, when the angle of the pole gets out of the range$\pm 12°$ or after

500 steps. The agent gets a reward of 1 for every step. Observations are arrays containing 4 values: the cart's position and velocity and the pole's angle and angular velocity. The action space is one discrete number taking the values 0 or 1 to push the cart left or right respectively.

The second environment studied here is acrobot. Like cartpole it is part of the classic control suite of gymnasium, it simulates a system of two links connected by a joint and puts the agent in control of the joint. This environment tasks the agent with reaching a line above it by moving the joint, its action space to do so is discrete and allows the agent to apply torques of -1, 0 or 1 on the joint. The observation space is the cosine and sine of the joint that locks the chain in place and the one controlled by the agent along with their respective angular velocity. The agent is given 100 steps to reach its goal, each step results in a reward of -1 and reaching the goal results in termination with a reward of 0.

The third environment is lunar lander. This environment is part of the box2d suite and is more complex. It puts the agent in control of a two-legged spaceship and tasks it with landing between two flags. The observation space is an 8-dimensional vector containing the ship's coordinates, linear and angular velocities and boolean values representing whether each leg is in contact with the ground. The action space is discrete and consists of doing nothing and firing the left, right or main engine. Rewards are increased for being close to the landing pad, moving slowly, having a leg in contact with the ground and landing safely. Rewards are decreased for being far from the landing pad, moving too fast, crashing, being tilted or firing the engines.

3.2. Cartpole

In this experiment PPO is compared to 4 other algorithms, a DQN, a REINFORCE algorithm, an actor critic (AC) and an advantage actor critic (A2C). The AC's learning rate is 0.00005 for the actor and 0.0001 for the critic, it uses 2 hidden layers with 128 units in each and , like all algorithms here, a gamma of 0.99. For A2C, both actor and critic use a learning rate of 0.0007 and are separate neural networks with 2 hidden layers with 64 units each. Because of exploding gradients issues, gradients are clipped to have norm up to 0.5. The REINFORCE network has two hidden layers of 128 and 64 units, it uses a learning rate of 0.001. The DQN uses experience replay and a target network. Its network has 2 hidden layers of 128 units each. It uses an epsilon-greedy strategy with an epsilon decaying epsilon starting at 0.9. The network is updated every 10 steps and the target network is updated every 200 steps.

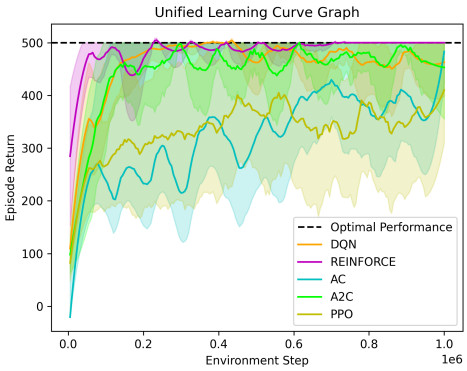

Figure 1. Learning curves of DQN, REINFORCE, Actor Critic and Advantage Actor Critic over 5 runs of 1 million steps of training in the cart-pole environment. REINFORCE converges to optimal performance the fastest and performs stably throughout training. AC and PPO converge the slowest and have the largest standard deviation. DQN and A2C both reach optimal performance a little after REINFORCE and perform relatively stably.

All algorithms appear to reach optimal performance in at least some runs. PPO appears to converge quite slowly, from review of the training logs it appears this is caused by very large swings in performance. The algorithm reaches optimal performance multiple times in every run but performance degrades drastically, leading to a poor average performance. This is also evidenced by the very large standard deviation. DQN, A2C and REINFORCE all converge very fast and are quite stable as evidenced by their small standard deviation. Actor critic also appears to converge quite slowly and has a very large standard deviation but is clearly learning as its performance consistently improves consistently.

3.3. Other Environments

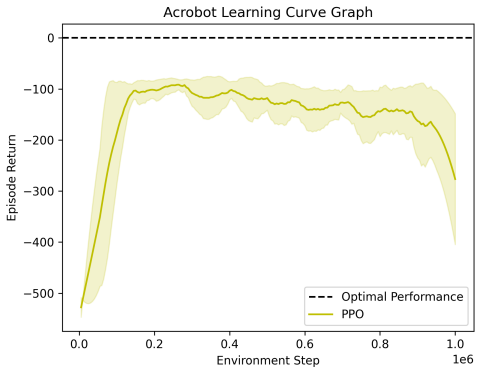

Figure 2. Learning curve of the PPO agent on Acrobot-v1, averaged over 5 independent runs. The solid line indicates the smoothed average return and the shaded region represents one standard deviation. Despite a very swift improvement in performance at the start the agent never reaches optimal performance. Performance is quite stable as standard deviation is small.

As can be observed in 2, the PPO agent is able to learn in various environments. It quickly improves at the start but plateaus around -100 and even deteriorates towards the end of the experiment, never reaching optimal performance. It can also be observed that performance is quite stable in this environment as the standard deviation is very small throughout the whole experiment.

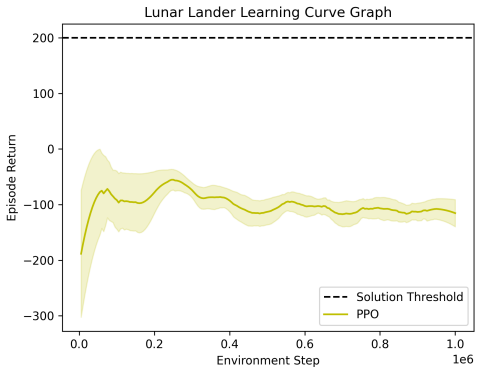

Figure 3. Learning curve of the PPO agent on LunarLander-v3, averaged over 5 independent runs. The solid line indicates the smoothed average return and the shaded region represents one standard deviation. The agent appears to learn during the first 200,000 timesteps but performance stabilizes and plateaus after that. The agent never approaches optimal performance.

As can be observed in 3 this configuration of PPO is unable to solve the lunar lander environment. However it must be noted that performance improves significantly during training, going from -300 to around -100. This shows that the agent is able to learn some elements of the environment but not enough to solve it, plateauing after around 200,000 timesteps. The agent is also relatively stable, even during the most unstable portion of training the standard deviation is smaller than at any point during cartpole training.

Discussion

PPO learns well in both acrobot and cartpole. While it doesn't quite reach optimal performance in acrobot, its performance improves over time. In cartpole it is able to reach optimal performance in all runs although it degrades quickly. This performance was reached after quite some time was spent tuning hyperparameters for better mean performances, showing how hyperparameter sensitive PPO can be. PPO also shows some learning in lunar lander (although quite minimal). This and the results in the acrobot environment are proof that PPO is able to perform in a wide range of environments. It also makes us believe that with more tuning and using configurations specific to each environment PPO would be able to solve all 3 environments studied here.

Additionally, the mean performance of PPO in cartpole shows the importance of early stopping. Many configurations tested reached optimal performance quickly but degraded a lot later, leading to poor mean performance. Implementing early stopping to stop training once optimal performance is reached would be quite helpful.

Learning Dynamics Comparison

An important aspect observed in the experiments is the different convergence behaviors and variance characteristics of PPO, A2C, and REINFORCE. REINFORCE as a Monte Carlo policy gradient method, converges quickly in the Cartpole environment and exhibits stable performance throughout training. This is likely due to its use of full-scene returns, which, despite having larger variance, works well in this low-dimensional, dense reward task.

In contrast, A2C uses temporal difference learning and bootstrapping, which reduces variance and achieves fast convergence. However, A2C exhibits some sensitivity to learning rate and weight initialization, which may explain occasional instabilities in early training.

PPO is expected to outperform A2C by limiting policy updates through clipping. However, PPO in Cartpole exhibits larger fluctuations and slower convergence. We assume that this is due to the smaller action space and faster learning dynamics of Cartpole, causing PPO's conservative update mechanism to become too restrictive. Therefore, PPO may benefit more in environments with continuous action spaces or sparse reward signals, where stability is more critical.

Overall, these differences highlight the trade-offs between fast adaptation (REINFORCE), variance reduction (A2C), and safe exploration (PPO). The results emphasize the importance of algorithm selection and hyperparameter tuning for specific environments.

Conclusion

This study showed that PPO is a powerful algorithm, it is able to learn (to some extent) in varying environments. However it is also quite hyperparameter sensitive and needs a lot of tuning to perform well consistently. This shows that depending on the environment's complexity, simpler algorithms may be a good solution. This also highlights the importance of early stopping and other ways to prevent performance degradation once optimality has been achieved.

Finally, the large swings in performance of PPO are quite surprising considering its core idea is to avoid changing action probabilities too much between training runs. The comparatively very small performance variations of the algorithm in other environments suggests this is caused by the cartpole environment rather than the algorithm. Investigating the cause of these wide swings in performance would be an interesting endeavor.

References

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Sutton, R. S. and Barto, A. G. *Reinforcement learning: An introduction*. MIT press, 2018.